

```

1  import numpy as np
2  from scipy.stats import norm
3
4  RUNS      = 5000
5  ALPHA     = 0.05
6  P         = 50
7  K         = 3
8  N_I       = 25
9  SEED      = 20250831
10 EPS       = 1e-12
11
12 def _Bi_within(X):
13     n = X.shape[0]
14     if n < 2:
15         return 0.0
16     G = X @ X.T
17     np.fill_diagonal(G, 0.0)
18     s = np.sum(G * G)
19     return s / (n * (n - 1))
20
21 def _Bij_cross(X, Y):
22     H = X @ Y.T
23     s = np.sum(H * H)
24     return s / (X.shape[0] * Y.shape[0])
25
26 def ahmad_Tg_test(data_groups, alpha=ALPHA):
27     g = len(data_groups)
28     n_i = [grp.shape[0] for grp in data_groups]
29     p = data_groups[0].shape[1]
30
31     B_within = [ _Bi_within(grp) for grp in data_groups ]
32     B_pairs = {}
33     for i in range(g):
34         for j in range(i+1, g):
35             Bij = _Bij_cross(data_groups[i], data_groups[j])
36             B_pairs[(i, j)] = Bij
37
38     sum_Bi = float(np.sum(B_within))
39     sum_Bij = float(np.sum(list(B_pairs.values())))
40     Tg = ((g - 1) * sum_Bi - 2.0 * sum_Bij) / (p * p)
41
42     P_star = 0.0
43     numerator_C3 = 0.0
44     for i in range(g):
45         for j in range(g):
46             if i == j:
47                 continue
48             ni, nj = n_i[i], n_i[j]
49             key = (i, j) if i < j else (j, i)
50             Bij = B_pairs[key]
51             numerator_C3 += ni * nj * Bij
52             P_star += ni * nj
53     C3 = numerator_C3 / max(P_star, EPS)
54
55     term1 = (g - 1) ** 2 * np.sum([1.0 / (ni ** 2) for ni in n_i])
56     term2 = 0.0
57     for i in range(g):
58         for j in range(i+1, g):
59             term2 += 2.0 / (n_i[i] * n_i[j])
60     A = term1 + term2
61
62     sigma_hat = 2.0 * np.sqrt(max(A, 0.0)) * (C3 / (p * p))
63     sigma_hat = max(sigma_hat, EPS)
64
65     Z = Tg / sigma_hat
66     crit = norm.ppf(1 - alpha)
67     return Z, crit, 0
68
69 #AR(1)
70 def ar1_matrix(p, rho):
71     idx = np.arange(p)
72     return rho ** np.abs(np.subtract.outer(idx, idx))
73
74 def simulate_groups_AR1_H0(p=P, k=K, n_i=N_I, rng=None):
75     if rng is None:

```

```

76     rng = np.random.default_rng()
77     F0 = ar1_matrix(p, rho=0.7)
78     means = [np.zeros(p) for _ in range(k)]
79     return [rng.multivariate_normal(means[i], F0, n_i) for i in range(k)]
80
81 def simulate_groups_AR1_H1_diff(p=P, k=K, n_i=N_I, rng=None):
82     if rng is None:
83         rng = np.random.default_rng()
84         F0 = ar1_matrix(p, rho=0.7)
85         F1 = ar1_matrix(p, rho=0.9)
86         means = [np.zeros(p) for _ in range(k)]
87         groups = []
88         for i in range(k):
89             Sigma_i = F1 if i == (k - 1) else F0
90             groups.append(rng.multivariate_normal(means[i], Sigma_i, n_i))
91     return groups, (k - 1)
92
93 #CS
94 def compound_symmetry_matrices(p):
95     I = np.eye(p); J = np.ones((p, p))
96     return 0.99 * I + 0.01 * J, 0.90 * I + 0.10 * J
97
98 def simulate_groups_CS_H0(p=P, k=K, n_i=N_I, rng=None):
99     if rng is None:
100         rng = np.random.default_rng()
101         C0, _ = compound_symmetry_matrices(p)
102         means = [np.zeros(p) for _ in range(k)]
103         return [rng.multivariate_normal(means[i], C0, n_i) for i in range(k)]
104
105 def simulate_groups_CS_H1_diff(p=P, k=K, n_i=N_I, rng=None):
106     if rng is None:
107         rng = np.random.default_rng()
108         C0, C1 = compound_symmetry_matrices(p)
109         means = [np.zeros(p) for _ in range(k)]
110         groups = []
111         for i in range(k):
112             Sigma_i = C1 if i == (k - 1) else C0
113             groups.append(rng.multivariate_normal(means[i], Sigma_i, n_i))
114     return groups, (k - 1)
115
116 #SIM
117 def simulate_groups_SIM_H0(p=P, k=K, n_i=N_I, rng=None):
118     if rng is None:
119         rng = np.random.default_rng()
120         S0 = np.eye(p)
121         means = [np.zeros(p) for _ in range(k)]
122         return [rng.multivariate_normal(means[i], S0, n_i) for i in range(k)]
123
124 def simulate_groups_SIM_H1_diff(p=P, k=K, n_i=N_I, rng=None, S1=None):
125     if rng is None:
126         rng = np.random.default_rng()
127         S0 = np.eye(p)
128         if S1 is None:
129             u = rng.uniform(2.0, 3.0, size=p)
130             S1 = np.diag(u)
131         means = [np.zeros(p) for _ in range(k)]
132         groups = []
133         for i in range(k):
134             Sigma_i = S1 if i == (k - 1) else S0
135             groups.append(rng.multivariate_normal(means[i], Sigma_i, n_i))
136     return groups, (k - 1)
137
138 #BTOEP
139 def toeplitz_matrix(p):
140     T0 = np.zeros((p, p)); T1 = np.zeros((p, p))
141     np.fill_diagonal(T0, 1.0)
142     i = np.arange(p - 1)
143     T0[i, i + 1] = T0[i + 1, i] = -0.5
144     np.fill_diagonal(T1, 1.0)
145     T1[i, i + 1] = T1[i + 1, i] = -0.05
146     return T0, T1
147
148 def simulate_groups_TOEP_H0(p=P, k=K, n_i=N_I, rng=None):
149     if rng is None:
150         rng = np.random.default_rng()
151         T0, _ = toeplitz_matrix(p)
152         means = [np.zeros(p) for _ in range(k)]
153         return [rng.multivariate_normal(means[i], T0, n_i) for i in range(k)]

```

```

154
155 def simulate_groups_TOEP_H1_diff(p=P, k=K, n_i=N_I, rng=None):
156     if rng is None:
157         rng = np.random.default_rng()
158     T0, T1 = toeplitz_matrix(p)
159     means = [np.zeros(p) for _ in range(k)]
160     groups = []
161     for i in range(k):
162         Sigma_i = T1 if i == (k - 1) else T0
163         groups.append(rng.multivariate_normal(means[i], Sigma_i, n_i))
164     return groups, (k - 1)
165
166 #VC
167 def variance_components(p, rng=None):
168     if rng is None:
169         rng = np.random.default_rng()
170     return np.diag(rng.uniform(1, 2, size=p)), np.diag(rng.uniform(3, 4,
171 size=p))
172
173 def simulate_groups_VC_H0(p=P, k=K, n_i=N_I, rng=None, A0=None):
174     if rng is None:
175         rng = np.random.default_rng()
176     if A0 is None:
177         A0, _ = variance_components(p, rng=rng)
178     means = [np.zeros(p) for _ in range(k)]
179     return [rng.multivariate_normal(means[i], A0, n_i) for i in range(k)]
180
181 def simulate_groups_VC_H1_diff(p=P, k=K, n_i=N_I, rng=None, A0=None, A1=None):
182     if rng is None:
183         rng = np.random.default_rng()
184     if (A0 is None) or (A1 is None):
185         A0_new, A1_new = variance_components(p, rng=rng)
186         A0 = A0 if A0 is not None else A0_new
187         A1 = A1 if A1 is not None else A1_new
188     means = [np.zeros(p) for _ in range(k)]
189     groups = []
190     for i in range(k):
191         Sigma_i = A1 if i == (k - 1) else A0
192         groups.append(rng.multivariate_normal(means[i], Sigma_i, n_i))
193     return groups, (k - 1)
194
195 #Monte Carlo
196 def monte_carlo(runs=RUNS, alpha=ALPHA, p=P, k=K, n_i=N_I, seed=SEED,
197 structure="AR1"):
198     rng = np.random.default_rng(seed)
199     S1_fix = None
200     A0_fix = None
201     A1_fix = None
202     if structure == "SIM":
203         u_fix = rng.uniform(2.0, 3.0, size=p)
204         S1_fix = np.diag(u_fix)
205     elif structure == "VC":
206         A0_fix, A1_fix = variance_components(p, rng=rng)
207
208     reject_H0_false = 0
209     reject_H0_true = 0
210
211     for _ in range(runs):
212         if structure == "AR1":
213             groups0 = simulate_groups_AR1_H0(p=p, k=k, n_i=n_i, rng=rng)
214             stat0, crit0, _ = ahmad_Tg_test(groups0, alpha=alpha)
215             if stat0 > crit0: reject_H0_false += 1
216
217             groups1, _ = simulate_groups_AR1_H1_diff(p=p, k=k, n_i=n_i, rng=rng)
218             stat1, crit1, _ = ahmad_Tg_test(groups1, alpha=alpha)
219             if stat1 > crit1: reject_H0_true += 1
220
221         elif structure == "CS":
222             groups0 = simulate_groups_CS_H0(p=p, k=k, n_i=n_i, rng=rng)
223             stat0, crit0, _ = ahmad_Tg_test(groups0, alpha=alpha)
224             if stat0 > crit0: reject_H0_false += 1
225
226             groups1, _ = simulate_groups_CS_H1_diff(p=p, k=k, n_i=n_i, rng=rng)
227             stat1, crit1, _ = ahmad_Tg_test(groups1, alpha=alpha)
228             if stat1 > crit1: reject_H0_true += 1
229
230         elif structure == "SIM":

```

```

229     groups0 = simulate_groups_SIM_H0(p=p, k=k, n_i=n_i, rng=rng)
230     stat0, crit0, _ = ahmad_Tg_test(groups0, alpha=alpha)
231     if stat0 > crit0: reject_H0_false += 1
232
233     groups1, _ = simulate_groups_SIM_H1_diff(p=p, k=k, n_i=n_i,
234         rng=rng, S1=S1_fix)
235     stat1, crit1, _ = ahmad_Tg_test(groups1, alpha=alpha)
236     if stat1 > crit1: reject_H0_true += 1
237
238     elif structure == "TOEP":
239         groups0 = simulate_groups_TOEP_H0(p=p, k=k, n_i=n_i, rng=rng)
240         stat0, crit0, _ = ahmad_Tg_test(groups0, alpha=alpha)
241         if stat0 > crit0: reject_H0_false += 1
242
243         groups1, _ = simulate_groups_TOEP_H1_diff(p=p, k=k, n_i=n_i,
244             rng=rng)
245         stat1, crit1, _ = ahmad_Tg_test(groups1, alpha=alpha)
246         if stat1 > crit1: reject_H0_true += 1
247
248     elif structure == "VC":
249         groups0 = simulate_groups_VC_H0(p=p, k=k, n_i=n_i, rng=rng,
250             A0=A0_fix)
251         stat0, crit0, _ = ahmad_Tg_test(groups0, alpha=alpha)
252         if stat0 > crit0: reject_H0_false += 1
253
254         groups1, _ = simulate_groups_VC_H1_diff(p=p, k=k, n_i=n_i, rng=rng,
255             A0=A0_fix, A1=A1_fix)
256         stat1, crit1, _ = ahmad_Tg_test(groups1, alpha=alpha)
257         if stat1 > crit1: reject_H0_true += 1
258
259     return {
260         "structure": structure,
261         "alpha": alpha,
262         "runs": runs,
263         "df": 0,
264         "empirical_size": reject_H0_false / runs,
265         "empirical_power": reject_H0_true / runs,
266         "beta_global": 1 - (reject_H0_true / runs),
267     }
268
269     if __name__ == "__main__":
270         for struct in ["AR1", "CS", "SIM", "TOEP", "VC"]:
271             res = monte_carlo(structure=struct)
272             print(f"==== Monte Carlo ({res['structure']}) structure) =====")
273             print(f"alpha = {res['alpha']} | runs = {res['runs']}")
274             print(f"Empirical size : {res['empirical_size']:.4f}")
275             print(f"Empirical power: {res['empirical_power']:.4f}")
276             print(f"Beta : {res['beta_global']:.4f}\n")

```

```

==== Monte Carlo (AR1 structure) ====
alpha = 0.05 | runs = 5000
Empirical size : 0.0736
Empirical power: 0.8476
Beta : 0.1524

```

```

==== Monte Carlo (CS structure) ====
alpha = 0.05 | runs = 5000
Empirical size : 0.0612
Empirical power: 0.5700
Beta : 0.4300

```

```

==== Monte Carlo (SIM structure) ====
alpha = 0.05 | runs = 5000
Empirical size : 0.0614
Empirical power: 0.9992
Beta : 0.0008

```

```

==== Monte Carlo (TOEP structure) ====
alpha = 0.05 | runs = 5000
Empirical size : 0.0626
Empirical power: 0.5898
Beta : 0.4102

```

```

==== Monte Carlo (VC structure) ====
alpha = 0.05 | runs = 5000
Empirical size : 0.0660
Empirical power: 0.9908
Beta : 0.0092

```